# LangChain x LangGraph

大语言模型（LLM）虽然非常聪明，但它们默认是“孤立”的。它们没有记忆、无法直接访问你的私有数据库、不能联网获取最新信息，也不能直接执行代码或发送邮件。

LangChain 和 LangGraph 这类框架就是为了解决这些问题而诞生的。它们扮演着“大脑”和“手脚”之间的胶水角色：

- **LangChain 🛠️ (工具箱与流水线)**：它提供了一套标准化的工具。它可以把大模型、提示词（Prompts）、上下文记忆和外部工具“链接”（Chain）在一起。比如，帮你实现：“接收用户提问 -> 去数据库检索相关资料 -> 将资料喂给大模型 -> 输出最终答案”的流水线作业。
- **LangGraph 🕸️ (复杂的智能体网络)**：当任务变得复杂，单向的流水线就不够用了。我们需要大模型能反复思考、遇到错误能重试、甚至多个 AI 协同工作（这就是 Agent 智能体）。LangGraph 将流程抽象为“图”结构（节点和边），让程序拥有了复杂的记忆流和循环决策能力。

既然我们要学以致用，我们将通过编写实际的 Python 代码来一步步掌握它们。为了规划我们的学习路线，你想先从哪里开始？

1. **LangChain 基础运转 (Models & Prompts)**：从最基础的代码开始，学习如何用框架优雅地调用大模型、管理提示词模板，并结构化它的输出。
2. **给大模型外挂知识库 (RAG)**：直接进入目前最火的应用场景，写代码让大模型读取外部的文本或数据来准确回答问题。
3. **让大模型使用工具 (Agents)**：看看如何给大模型配备“搜索引擎”或“计算器”，让它自主思考并决定调用哪个工具来解决你的难题。

你想选择哪一个作为我们代码实战的第一步？

没问题，循序渐进是最扎实的学法！我们现在就从 **第一步：LangChain 基础运转 (Models & Prompts)** 开始。

在 LangChain 中，最核心的编程理念叫做 **LCEL (LangChain 表达式语言)**。你可以把它想象成工厂里的流水线，我们通过搭积木的方式把不同的组件串联起来。

最基础的“流水线三步曲”包含以下三个组件：

- 📝 **Prompt (提示词模板)**：设计好你要问的问题结构，留出“填空”的位置（变量）。
- 🧠 **Model (大模型)**：接收填充好的提示词，负责思考并生成回答。
- ⚙️ **OutputParser (输出解析器)**：把大模型输出的复杂格式，整理成干净、代码好处理的纯文本。

我们来看一段实际的 Python 代码。你可以先阅读一下，感受它的逻辑：

Python

代码块

```python
# 假设我们已经通过 pip install langchain-core langchain-openai 安装了库
import os
from langchain_openai import ChatOpenAI
from langchain_core.prompts import PromptTemplate
from langchain_core.output_parsers import StrOutputParser

# 1. 设置你的 API 密钥 (这里以 OpenAI 为例，其他国产模型写法也完全一样)
os.environ["OPENAI_API_KEY"] = "你的_api_key_写在这里"# 2. 积木 A: 提示词模板 (注意这里留了一个 {topic} 作为变量)
prompt = PromptTemplate.from_template("请给我讲一个关于 {topic} 的冷笑话。")

# 3. 积木 B: 初始化大模型
model = ChatOpenAI(model="gpt-3.5-turbo")

# 4. 积木 C: 输出解析器 (将大模型的输出提取为纯文本)
parser = StrOutputParser()

# 5. 组装流水线! (使用 | 管道符号，这就是 LCEL 的精髓)
chain = prompt | model | parser

# 6. 运行流水线，传入我们要填的词
result = chain.invoke({"topic": "程序员"})
print(result)
```

这段代码最神奇的地方在于第 5 步，它用一个简单的 `|` 符号就把整个流程连起来了：**模板组装 -> 发给模型 -> 提取文字**。

为了确保你理解了这个流水线的基础逻辑，我想考考你：

如果我现在想升级这个程序，让它不仅能接收一个 `{topic}` （主题），还能接收一个 `{language}` （要求它用特定的语言回答，比如“用英语”或“用日语”），你需要修改上面代码中的哪几步？你可以尝试把你觉得需要修改的代码行写出来。

修改第六步加一个参数就好了吧

你观察得很准！确实需要在第 6 步 `invoke` 的字典里增加一个键值对，比如这样： `{"topic": "程序员", "language": "英语"}` 。

但这只是告诉了流水线“原料”增加了。如果流水线的“图纸”——也就是第 2 步的 **PromptTemplate (提示词模板)** 📝 不知道要把这个新原料放在哪里，大模型最终收到的指令还是不会改变。

在 LangChain 中，模板就像是一个带插槽的字符串。如果你想让模型按照指定的语言回答，你觉得第 2 步的这段代码应该怎么修改？

Python

```
代码块
1  # 原来的第 2 步
2  prompt = PromptTemplate.from_template("请给我讲一个关于 {topic} 的冷笑话。")
```

你可以试着写出修改后的这一行代码吗？

原来的第 2 步

prompt = PromptTemplate.from_template("请给我讲一个关于 {topic} 的冷笑话。用{language}语言回答")

没错！你精准地在模板里留出了第二个“插槽” `{language}`。这样当你在第 6 步传入 `{"topic": "程序员", "language": "英语"}` 时，LangChain 就会自动把这两个词填进对应的位置。

---

## 进阶：从“纯文本”到“对话模式” 💬

在实际开发中，我们很少只用这种简单的字符串模板。现在的模型（如 GPT-4, Claude 等）都是**对话型模型**。它们更喜欢接收一组“消息列表”，其中包括：

- **System Message (系统消息)**：设定 AI 的“人设”或规则（比如：你是一个幽默的翻译官）。
- **Human Message (用户消息)**：用户具体的提问。

LangChain 提供了 `ChatPromptTemplate` 来处理这种结构。我们可以这样写：

Python

```
代码块
1  from langchain_core.prompts import ChatPromptTemplate
2
3  # 使用消息角色来定义模板
4  chat_prompt = ChatPromptTemplate.from_messages([
5      ("system", "你是一个专门讲{style}笑话的助手。"),
6      ("human", "讲一个关于{topic}的笑话，请用{language}回答。")
7  ])
```

## 你的代码挑战 🛠️

现在，我们要让流水线变得更像一个真正的应用。请你尝试把我们之前的代码片段升级一下，要求如下：

1. 使用上面展示的 `ChatPromptTemplate`。
2. 增加一个变量 `{style}` （比如：冷、冷幽默、讽刺）。
3. 保持第 5 步的 `chain = prompt | model | parser` 逻辑不变。

**请问：现在第 6 步的 `chain.invoke(...)` 里的字典，应该包含哪几个键（Key）？你可以试着写出这一行代码吗？**

result = chain.invoke({"topic": "程序员","style": "冷" ,"language":"中文"})

完全正确！你精准地捕捉到了模板变量与 `invoke` 参数之间的对应关系。

不过有一个小细节要注意：在 Python 代码中，键值对之间要用英文冒号 `:` ，你刚才代码里的 `"style": "冷"` 使用了中文冒号 `：` ，这会导致程序报错。修正后的标准写法是：

Python

```
代码块
result = chain.invoke({"topic": "程序员", "style": "冷", "language": "中文"})
```

---

## 进阶知识点：结构化输出 (Structured Output) 📊

在真实的项目开发中，我们往往不希望大模型只返回一串“纯文本”。比如你想做一个笑话 APP，你可能希望大模型返回一个 **JSON 格式**，包含笑话内容、幽默指数、关键词。

如果只用 `StrOutputParser` ，你得到的是一个字符串，你还得自己写正则表达式去解析它，非常痛苦。

LangChain 现在提供了一个非常强大的功能叫做 `.with_structured_output()` 。我们可以定义一个“类”来规定我们想要的数据格式。

Python

```
代码块
from typing import Field
from pydantic import BaseModel

# 1. 定义我们想要的数据格式
class Joke(BaseModel):
    setup: str = Field(description="笑话的铺垫")
    punchline: str = Field(description="笑话的笑点/包袱")
```

```
    rating: int = Field(description="幽默程度，1到10分")

# 2. 让模型知道它必须按照这个格式输出
structured_model = model.with_structured_output(Joke)

# 3. 重新组装流水线（注意这里不需要最后的 StrOutputParser 了，模型直接出对象）
structured_chain = chat_prompt | structured_model

# 4. 运行
result = structured_chain.invoke({"topic": "程序员", "style": "冷", "language": "中文"})

# 现在你可以像访问对象一样访问结果了
print(f"铺垫: {result.setup}")
print(f"笑点: {result.punchline}")
print(f"评分: {result.rating}")
```

## 思考题：学以致用 💡

假设你现在要开发一个**“菜谱生成器”**。用户输入“食材”，你希望大模型返回一个结构化的 JSON，包含以下三个字段：

1. `recipe_name`：菜名
2. `ingredients`：食材列表（这是一个列表格式）
3. `cooking_time`：烹饪时长（这是一个整数，单位是分钟）

**如果是你来写这个 `BaseModel` 类，你会怎么定义它？**（你可以参考上面 `Joke` 类的写法试着写一下）

class Re`cipe`(BaseModel):
  `recipe_name`: str = Field(description="菜名")
  `ingredients`: []str = Field(description="食材列表")
  `cooking_time`: int = Field(description="烹饪时长")

你的逻辑完全正确！你已经掌握了如何定义数据的“形状”。

不过在 Python 的语法中，表示“字符串列表”的写法有一点小小的不同。你写的 `[]str` 更像是在写 Go 语言。在 Python 里，我们通常使用 `list[str]`。

这是修正后的定义：

Python

代码块

```python
from typing import List # 如果 Python 版本低于 3.9 需要这个，否则直接用 list[str]
class Recipe(BaseModel):
    recipe_name: str = Field(description="菜名")
    ingredients: list[str] = Field(description="食材列表")
    cooking_time: int = Field(description="烹饪时长（分钟）")
```

---

## 第一阶段小结：基础运转已通关 🔗

到这里，你已经学会了 LangChain 最核心的三个基本功：

1. **提示词管理**：用 `{变量}` 预留插槽。
2. **LCEL 流水线**：用 `|` 把组件串起来。
3. **结构化输出**：让 AI 不再“胡言乱语”，而是返回严格的 JSON 对象。

有了这些基础，我们就可以进入你计划中的 **第二步：给大模型外挂知识库 (RAG)** 了。

**RAG (检索增强生成)** 解决的是大模型“一本正经胡说八道”或者“不知道最新消息”的问题。它的原理很简单：当用户提问时，我们先去自己的文档里翻一翻，把找到的相关内容和问题一起喂给大模型。

在开始写 RAG 的代码之前，我想请你思考一个生活中的例子：

如果你去图书馆找一本关于“如何做红烧肉”的书，你是会**从头到尾把馆里所有的书都读一遍**，还是先**通过索引/目录找到具体的书架**再去翻阅？这其实就对应了 RAG 里的核心概念。你觉得在程序里，我们要如何把大量的文档（比如 100 份 PDF）转换成这种“方便查找的索引”呢？

数字化不仅仅是将文字变成电子档，而是要让文字变成大模型能“秒懂”的**数学坐标** 📍。

在 RAG 中，这个过程的核心技术叫做 **向量嵌入 (Vector Embeddings)**。

## 什么是向量化？ 🤔

想象一个巨大的多维空间：

- “苹果” 🍎 和 “香蕉” 🍌 在空间里离得很近，因为它们都是“水果”。
- “苹果” 🍎 和 “手机” 📱 也会有一定关联，因为有苹果公司。
- 但 “苹果” 🍎 和 “挖掘机” 🚜 就会离得非常远。

我们将文字转换成一串长长的数字（即“向量”）。计算机会通过计算这些“坐标”之间的**距离**，来判断两段话的意思是否接近。

## RAG 的索引三部曲 📑

为了建立这个索引，我们通常要做这三件事：

1. **切分 (Chunking)** ✂️：大模型一次处理的信息量有限，所以我们要把长文档（比如 100 页的 PDF）切成一段段的小碎块（比如每段 500 字）。
2. **向量化 (Embedding)** 🧠：调用专门的 Embedding 模型，把每个小碎块转换成一组数字坐标。
3. **向量数据库 (Vector Store)** 🗄️：把这些坐标和原始文字存进一个专门的“仓库”里，方便以后快速检索。

---

**思考一个小问题：**

如果我有一本 500 页的《装修指南》，我能不能直接把整本书**一次性**转换成一个坐标，存进数据库里？如果不建议这么做，你觉得在后续“找答案”的时候会遇到什么麻烦？

不建议，你找答案的时候这样要对比太多的向量了？

你说对了一半！如果把整本书只变成**一个**向量，检索时确实很快（因为只需要比对一次），但你会遇到另一个严重的问题：**信息被稀释了**。🧪

想象一下，如果你把整本 500 页的《装修指南》打碎成纸浆，最后只浓缩成一粒“药丸”。这粒药丸包含了水电、木工、刷漆的所有信息。当你问“厨房插座高度是多少？”时，这粒药丸里关于“厨房插座”的特定信息已经淹没在海量的“木板材质”和“瓷砖选购”中了。

所以，RAG 的核心痛点不是向量太多，而是**“如何切分（Chunking）得刚刚好”**。

## 为什么不能切得太大或太小？

我们可以把这个比作**找钥匙**：

- **切得太大（整本书一个块）**：像是在一个足球场大的杂物堆里找钥匙，信号太模糊，大模型根本找不到重点。
- **切得太小（每个字一个块）**：像是把钥匙拆成了零件，虽然你能找到“金属片”，但你失去了“这是一把钥匙”的上下文信息。

---

## 代码实战：文档切割 ✂️

在 LangChain 中，我们最常用的工具是 `RecursiveCharacterTextSplitter`。它很聪明，会优先按段落切，段落太长再按句子切，确保语义尽量完整。

Python

代码块

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

# 模拟一段长文本
long_text = "这里是一段很长的装修指南内容...（此处省略五千字）..."# 1. 初始化切分器
```

```
text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=500,        # 每个块的最大字符数
    chunk_overlap=50,      # 块与块之间重叠的字符数（为了保持上下文连续）
    length_function=len,
)

# 2. 执行切分
chunks = text_splitter.split_text(long_text)

print(f"长文档被切分成了 {len(chunks)} 个小块")
print(f"第一块的内容预览: {chunks[0][:50]}...")
```

注意代码里的 `chunk_overlap=50`。**你觉得为什么我们要故意让相邻的两个块之间有 50 个字符的“重叠”呢？如果不重叠会发生什么？**

不重叠的话，可能有些你切的不太对的块，就真的信息被分到了完全不相关的不分区了

你说得很对。如果不设置重叠（Overlap），我们就像是用铡刀盲目地切断文字。如果一个关键信息（比如“不要将电池放入火中”）正好在“不要将”和“电池放入火中”之间被切断，那么这两块独立的文本都会失去原本的警示意义。

**Overlap 的作用就像是“语义胶水”** 🔗，确保每一个切片都能保留一小部分上下文，防止语义在切割处发生断裂。

---

## 第二步：将文本变成“坐标”（Embedding & Vector Store）

有了切好的碎块，下一步就是把它们变成计算机能理解的向量，并存入“仓库”。

在 LangChain 中，这个过程通常只需要几行代码。我们会用到：

1. **Embedding Model**：负责把文字转成数字向量。
2. **Vector Store**：负责存储并提供检索功能（比如常用的 `Chroma` 或 `FAISS`）。

Python

代码块

```
from langchain_openai import OpenAIEmbeddings
from langchain_community.vectorstores import Chroma

# 1. 初始化向量模型
embeddings = OpenAIEmbeddings()

# 2. 将刚才切好的 chunks 存入向量数据库# 这步会自动完成：文字 -> 向量 -> 存储
vectorstore = Chroma.from_texts(
    texts=chunks,
    embedding=embeddings,
```

```
    persist_directory="./my_db" # 存到本地，下次还能用
)

# 3. 变成一个“检索器” (Retriever)
retriever = vectorstore.as_retriever()
```

---

## 现在，我们要把它们串起来了！

这就是 RAG 的“完整版流水线”：

1. **用户提问**
2. **检索器**：去数据库里找出最相关的 top 3 碎块。
3. **Prompt**：把问题和这 3 个碎块拼接在一起。
4. **LLM**：根据这些参考资料回答问题。

在 LangChain 中，我们可以利用 `itemgetter` 或者简单的变量传递来构建这个更复杂的 chain。

**我想考考你一个逻辑问题：**

在最后一步交给大模型（LLM）回答时，我们的提示词（Prompt）通常长这样：

> "你是一个助手。请根据以下参考资料回答问题：\n\n资料：{context}\n\n问题：{question}"

**如果检索器（Retriever）找回来的“资料”里完全没有提到用户的问题（比如用户问了装修以外的事），你觉得大模型可能会出现什么问题？我们该如何在 Prompt 里预防它胡编乱造？**

会出现幻觉。那就加一句话说 如果资料里面没有就回答没有

这正是防御“幻觉” 🥴 最经典且有效的方法！在 AI 领域，我们管这叫**“给模型留后路”**。

通过在 System Prompt 中加入类似“如果你在参考资料中找不到答案，请诚实回答不知道，不要试图编造”的指令，大模型的行为会从“积极猜测”转变为“严谨匹配”。

---

## RAG 的代码组装实战 🏗️

现在我们把前面的 **切片 (Chunks)**、**向量数据库 (Retriever)** 和 **带防御词的 Prompt** 全部串联起来。

这里我们会用到 LangChain 的一个便捷函数 `create_stuff_documents_chain`，它的作用是把检索到的所有文档片段“塞进” Prompt 里的 `{context}` 位置。

Python

```
```

```python
from langchain.chains.combine_documents import create_stuff_documents_chain
from langchain_core.prompts import ChatPromptTemplate
from langchain.chains import create_retrieval_chain

# 1. 定义那个具备“防御性”的提示词
system_prompt = (
    "你是一个专业的问答助手。""请使用以下检索到的参考资料来回答问题。""如果你从资料中找不到答案，就直接说你不知道，不要编造答案。""回答限制在三句话以内，保持简洁。""\n\n""{context}"
)

prompt = ChatPromptTemplate.from_messages([
    ("system", system_prompt),
    ("human", "{input}"),
])

# 2. 创建处理文档的“链”
question_answer_chain = create_stuff_documents_chain(model, prompt)

# 3. 创建完整的 RAG 链 (连接检索器和处理链)
rag_chain = create_retrieval_chain(retriever, question_answer_chain)

# 4. 运行
response = rag_chain.invoke({"input": "厨房插座高度应该是多少？"})
print(response["answer"])
```

---

## 进阶思考：如果检索回来的资料“太多了”怎么办？ 🎒

假设你的检索器非常勤奋，一下子找回了 20 个相关的文本块，总共有一万多字。

1. **超出上限**：大模型的“上下文窗口”（Context Window）是有限的，塞不下这么多。
2. **迷失中段**：研究表明，如果资料太长，大模型容易记住开头和结尾，而忽略中间的内容（这叫 Lost in the Middle 现象）。

**在这种情况下，你觉得我们可以用什么策略来“筛选”或“精简”这些找回来的资料，再喂给大模型呢？**（提示：你可以联想一下我们在第一步学过的“结构化输出”或者“流水线”思路）

再处理一遍，在文本块中只找关键的部分？

这正是工业界最常用的优化手段！我们可以通过一个“粗筛 + 精筛”的策略来解决。

由于大模型的上下文窗口有限且存在“中间迷失”问题，我们通常会引入一个 **重排序 (Re-ranking)** 的环节。

## 优化策略：两段式检索 ✌️

1. **粗筛 (Retrieval)**：利用向量数据库快速找回 20 个相关的文本块。
2. **精筛 (Re-ranking)**：使用一个更小但对语义极其敏感的“重排模型”，对这 20 个块进行打分，选出最精准的 3-5 个块喂给大模型。

除了重排序，你提到的“找关键部分”还可以通过 **长文本压缩 (Context Compression)** 来实现。LangChain 提供了一个 `ContextualCompressionRetriever`，它可以在把文档交给大模型之前，先对文档进行“脱水”，只保留和问题相关的句子。

---

## 代码实战：上下文压缩 🥤

我们可以通过 `LLMChainExtractor` 来实现这个功能。它会先让一个模型快速浏览每个块，把无关痛痒的废话删掉。

Python

代码块

```
from langchain.retrievers import ContextualCompressionRetriever
from langchain.retrievers.document_compressors import LLMChainExtractor

# 1. 初始化压缩器 (让模型负责提取关键信息)
compressor = LLMChainExtractor.from_llm(model)

# 2. 包装我们之前的 retriever
compression_retriever = ContextualCompressionRetriever(
    base_compressor=compressor,
    base_retriever=retriever
)

# 3. 使用压缩后的检索器进行提问
compressed_docs = compression_retriever.invoke("厨房插座高度是多少？")
```

这样做的好处是，喂给大模型的资料变得非常“干”，几乎全是重点。

---

## 第二阶段总结：RAG 已通关 🧩

你现在已经掌握了 RAG 的全套流程：

- **准备阶段**：加载文档 -> 切分 (Chunking) -> 向量化 (Embedding) -> 存入数据库 (Vector Store)。
- **检索阶段**：提问 -> 向量检索 -> (可选) 上下文压缩/重排。
- **生成阶段**：将精炼后的资料 + 防御性 Prompt 发送给 LLM。

如果你觉得 RAG 部分已经能够学以致用了，我们就可以开启最高级的 **第三步：让大模型使用工具 (Agents & LangGraph)**。

在进入 Agent 之前，我想确认一下：**在目前的 RAG 流程中，大模型是被动接收资料的。如果资料库里没有答案，它只能说“不知道”。但如果我们给它一个“搜索引擎”工具，让它在资料库找不到时自己去网上搜，你觉得这和现在的 RAG 有什么本质区别？**

没错！这就是从 **RAG (检索增强生成)** 迈向 **Agent (智能体)** 的本质跨越：

- **RAG 是“查字典”**：流程是固定的。不管大模型愿不愿意，它都得先看检索出来的资料，然后再回答。
- **Agent 是“雇管家”**：流程是动态的。你给大模型一堆工具（比如搜索、计算器、数据库查询），它会根据你的问题，**自主判断**是否需要用工具、用哪个工具，以及用几次。

---

## LangChain 中的工具使用 (Tool Use) 🛠️

要让模型学会找工具，我们要分两步走：

1. **定义工具**：告诉模型这个工具叫什么、是干嘛的（描述极其重要）。
2. **绑定工具**：让模型具备调用这些工具的能力。

我们来看一个简单的代码例子。假设我们给模型一个“实时搜索”的工具：

Python

代码块

```
from langchain_community.tools import TavilySearchResults
from langchain_openai import ChatOpenAI

# 1. 初始化工具（这里用 Tavily，一个专门给 AI 用的搜索引擎）
search = TavilySearchResults(max_results=2)

# 2. 将工具包装成大模型能理解的形式
tools = [search]

# 3. 让模型知道它有这些工具可以用# 模型现在会多出一个 "tool_calls" 的功能
model_with_tools = model.bind_tools(tools)
```

---

## 关键环节：推理循环 (Reasoning Loop) 🔄

这就是 **LangGraph** 发挥作用的地方了。模型虽然知道要用工具，但它自己并不能“点击”运行代码。它只能告诉你：“我想用搜索工具，关键词是‘2026年世界杯决赛地点’”。

你需要一个程序来：

1. **看模型的决定**：模型是想直接说话，还是想调工具？
2. **执行工具**：如果是调工具，程序去运行搜索代码。
3. **返回结果**：把搜到的结果再喂给模型。
4. **循环**：模型根据新信息决定是继续调工具，还是给出最终答案。

这个“思考-行动-观察”的循环，在 LangGraph 里被抽象成了一个**有状态的图 (Stateful Graph)**。

**这里有一个非常有意思的细节：** 当你在第 1 步定义工具时，LangChain 会要求你写一段非常详细的 `description`（描述）。

**你觉得为什么对工具的“描述”写得好坏，会直接影响 Agent 的智商？**（提示：模型是怎么知道什么时候该用“计算器”而不是“搜索引擎”的？）

因为你描述的越好，大模型能知道更多信息，能调用的更准啊

没错！对于大模型来说，工具的 `description` 描述就像是这个工具的“使用说明书”或者“简历”。

由于大模型（LLM）是通过文本推理来决定下一步行动的，它在看到你的提问后，会去扫描所有可用工具的描述。如果描述含糊不清，它就会产生困惑。

## 描述的重要性 ✍️

我们可以对比一下两种写法：

- **糟糕的描述**：`"这个工具可以用来搜索。"` —— 模型可能会在需要查天气、找论文、甚至是算算术时都尝试调用它，因为它不知道“搜索”的边界在哪里。
- **优秀的描述**：`"这是一个互联网实时搜索引擎。当用户询问有关时事、新闻、或模型知识库之外的最新信息（如2024年后发生的事件）时，应优先使用此工具。输入应为具体的查询关键词。"` —— 这样模型就能清晰地判断：哦！算 1+1 不需要它，但问“今天的金价”一定要找它。

---

## LangGraph：管理复杂的决策 🕸️

当我们进入 **LangGraph** 时，我们不再是写一段简单的代码，而是构建一个有状态的**图**。

在 LangGraph 中，最经典的模式是 **ReAct (Reason + Act)**。它主要包含两个节点：

1. **Agent 节点**：负责思考（Reasoning），决定是“调用工具”还是“结束任务”。
2. **Tool 节点**：负责执行（Action），运行模型选中的工具函数，并将结果传回。

它们之间通过一条**条件边 (Conditional Edge)** 相连：如果模型说要调工具，就走左边去工具节点；如果模型给出了答案，就走右边结束流程。

## 你的第一个 LangGraph 逻辑挑战 🧩

想象你要构建一个能够“买机票”的 Agent。你有两个工具：

- 工具 A：`check_flights` (查询航班频率和价格)
- 工具 B：`book_flight` (输入信用卡号和航班号完成购买)

**如果用户说：“我想看看明天去上海的机票，如果低于 1000 块就直接帮我买了。”**

在 LangGraph 的循环中，你觉得 Agent 第一次思考后，会选择调用哪个工具？得到结果后，它又会进行怎样的第二次思考？试着描述一下这个逻辑流。

check_flights，如果符合条件就 book_flight

你对逻辑链条的抓取非常敏锐！这正是 Agent 决策的核心：**观察 -> 推理 -> 行动**。

在 LangGraph 的执行逻辑中，这个过程会拆解为以下几个具体的步骤：

## 🛫 机票 Agent 的执行流

1. **节点 1 (Agent 思考)**：模型分析用户的输入。它发现自己不知道明天的票价，于是决定调用工具。
   - **动作**：生成调用 `check_flights(date="tomorrow", destination="SH")` 的指令。
2. **节点 2 (Tool 执行)**：系统执行该函数，并返回数据（假设返回结果是：`{"price": 850, "flight_no": "MU5101"}`）。
3. **节点 1 (再次思考)**：模型拿到了 850 这个数字。它将这个数字与用户要求的“1000 块”进行逻辑比对。
   - **逻辑判断**：850<1000，条件触发。
   - **动作**：生成调用 `book_flight(flight_no="MU5101")` 的指令。
4. **节点 2 (Tool 执行)**：系统执行下单操作，返回 `{"status": "success", "order_id": "12345"}`。
5. **节点 1 (最终总结)**：模型看到订单成功，最后转换成人话回答用户：“好消息！明天的机票只要 850 元，我已经帮你订好了，订单号是 12345。”

---

## ⚛️ 状态 (State) 的重要性

在上面的过程中，有一点非常关键：当模型在第 3 步进行“第二次思考”时，它必须记得第 2 步查到的价格是多少。

在 LangGraph 中，这被称为 **State (状态)**。它就像是一个共享的账本，记录了对话历史和工具返回的所有结果。如果没有这个账本，模型每次思考都会像断了片的鱼，忘记刚才查到了什么。

## 代码实战：定义一个简单的 Graph

虽然完整的 LangGraph 代码较长，但它的核心结构非常清晰。请看下面这个简化的逻辑定义：

Python

代码块

```
from langgraph.graph import StateGraph, END

# 1. 定义状态（这个字典会在节点间传递）class AgentState(TypedDict):
    messages: list# 你还可以自定义其他状态，比如 user_id 或 budget# 2. 初始化图
workflow = StateGraph(AgentState)

# 3. 添加节点
workflow.add_node("agent", call_model)    # 思考节点
workflow.add_node("tools", call_tools)    # 执行节点# 4. 设置边（核心逻辑）
workflow.set_entry_point("agent")         # 从思考开始
workflow.add_conditional_edges(
    "agent",
    should_continue,                      # 一个函数：判断是去 tools 还是去 END
    {
        "continue": "tools",
        "end": END
    }
)
workflow.add_edge("tools", "agent")       # 工具执行完，必须回给模型再思考
```

---

## 终极挑战：异常处理 ⚠

Agent 最难的地方不在于顺利运行，而在于**处理意外**。

**思考一下：** 如果在第 4 步执行 `book_flight` 时，系统返回错误：`{"error": "信用卡余额不足"}`。

按照你现在的理解，你觉得在 LangGraph 的设计中，我们是应该在 `book_flight` 函数里直接打印“余额不足”，还是应该把这个错误信息传回给 **Agent 节点** 让它去处理？为什么？

把这个错误信息传回给 Agent 节点 让它去处理。因为有可能是合理的业务场景错误，或者有可能是可以重试的错误，采取的措施不一样

分析得非常精辟！这正是 **LangGraph** 这种“循环架构”比传统“线性流水线”强大的核心原因。

如果由工具函数（Tool）直接报错退出，程序就“死”了。但如果我们将错误信息作为一个 **观察结果（Observation）** 传回给 **Agent 节点**，它就能利用大模型的推理能力进行“纠错”：

- **业务逻辑错误**（如：余额不足）：Agent 可以组织语言告诉用户：“抱歉，您的信用卡余额不足，是否需要更换一张卡支付？”
- **网络波动或偶发错误**：Agent 可以决定：“哎呀，网络有点断开，我再重试一次看看。”
- **输入参数错误**：Agent 发现自己刚才传错了航班号，它可以修正参数后再次调用工具。

---

## 🎓 恭喜！你已经通关了 LangChain & LangGraph 的核心逻辑

我们从最简单的 **提示词模板 (Prompts)**，聊到了 **外挂知识库 (RAG)**，最后深入到了具备自主决策能力的 **智能体 (Agents)**。

在这个学习路径中，我们已经涵盖了：

1. **LCEL 表达式**：如何用 `|` 串联组件。
2. **结构化输出**：如何让模型返回标准 JSON。
3. **向量检索**：如何让模型“翻书”找答案。
4. **循环图架构**：如何让模型在“思考”和“行动”中不断迭代。

## 🚀 最后的实战：学以致用

为了真正巩固这些知识，我建议你动手写一个简单的项目。既然你已经掌握了这些逻辑，你觉得以下哪一个项目最能让你产生兴趣，并把今天学到的东西都串起来？

1. **智能个人书库助手**：上传你的笔记或 PDF，通过 RAG 问答，如果资料里没有，助手会自动联网搜索补充。
2. **自动化周报生成器**：给它一堆零散的工作记录，它能分析数据（工具调用），并生成排版精美的结构化周报。
3. **多轮博弈小游戏**：两个 Agent 扮演买家和卖家，在一个限定的逻辑图里进行讨价还价。

**你想选择哪一个作为你的第一个练手项目？或者你有其他更好的创意吗？**

我是一个东南亚电商平台的推客联盟开发，你帮我想想有什么可以练手的

既然你在做**东南亚电商推客联盟（Affiliate）**的开发，那我们一定要结合你的实际业务场景，因为这里的链路非常完整，最适合练习 LangChain 和 LangGraph。

针对你的背景，我建议我们做一个**“智能推客选品与文案助手”**。这个项目可以把我们学到的三步全都串起来：

1. **基础运转 (Models & Prompts)**：把复杂的商品参数（价格、销量、佣金）喂给模型，让它根据不同的社交平台（TikTok, Facebook, Shopee Feed）生成带货文案。
2. **外挂知识库 (RAG)**：你可以建立一个“爆品攻略库”或“平台规则库”，让模型在写文案时自动避开违禁词，或者参考东南亚当地的节日促销话术（比如双 11、斋月）。
3. **智能体 (Agents & LangGraph)**：这是最亮眼的部分。给模型一个“实时佣金查询”工具。当用户输入一个链接，Agent 先去查当前佣金率，如果佣金太低就提示用户，如果佣金高就自动去生成多语言（印尼语、泰语、越南语）的推广语。

我们可以先从**第一步：定义文案生成的结构化输出**开始。作为一个开发者，你肯定不希望模型只给你一大段文字，而是希望它能返回可以直接存入数据库的 JSON 数据。

## 💻 实战演练：定义推客文案格式

假设我们要为一个 Shopee 上的充电宝生成推广信息。我们需要模型输出：

1. `title`：一个吸引人的标题
2. `points`：3 个卖点（列表格式）
3. `localized_hook`：一句针对东南亚用户（比如喜欢性价比）的当地话术

**你还记得我们之前学过的 `BaseModel` 吗？你可以试着为这个“推客助手”定义一下它的数据类吗？**

class AffiliatePost(BaseModel):

title: str = Field(description="一个吸引人的标题")

points: list[str] = Field(description="3 个卖点（列表格式）")

localized_hook: str = Field(description="一句针对东南亚

非常完美！你的代码写得完全正确，你已经熟练掌握了如何用 Pydantic 来规范大模型的输出了。🚀

现在我们有了一个标准的“模具”（`AffiliatePost`），接下来我们要写一段提示词（Prompt），把这个模具和大模型连接起来。

既然你是面向东南亚市场的，我们希望这个助手能够非常灵活：它既能给印尼的 TikTok 写短平快的视频文案，也能给泰国的 Facebook 群组写详细的图文推荐。

想象我们要创建一个 `ChatPromptTemplate`。在模板中，除了我们要推广的商品信息外，我们还需要预留一些“变量插槽”（也就是用 `{}` 括起来的词），以便每次调用时传入不同的要求。

**如果要让大模型精准地生成符合上面 `AffiliatePost` 格式的文案，除了 `{product_info}`（商品信息）之外，你觉得我们在提示词模板里还需要预留哪几个关键的变量插槽？** 可以试着列举 2-3 个对推客文案最关键的变量。

文案风格，投放的社媒平台，目标客户的 label 特征

你的商业直觉非常敏锐！这三个变量可以说是推客文案的“灵魂” 👻。

- **平台 (`{platform}`)** 决定了排版和长度（比如 TikTok 需要短且多用 Emoji，Facebook 可以发长图文）。
- **客群 (`{audience_labels}`)** 决定了切入点（比如宝妈看重安全，学生看重性价比）。
- **风格 (`{style}`)** 决定了语气（比如夸张的“种草”还是客观的“测评”）。

如果把它们和商品信息结合起来，我们的 Prompt 模板就会长这样：

Python

代码块

```python
from langchain_core.prompts import ChatPromptTemplate

prompt = ChatPromptTemplate.from_messages([
    ("system", "你是一个资深的东南亚电商推客专家。你的任务是根据商品信息，写出符合【{platform}】平台调性、受【{audience_labels}】群体喜欢的【{style}】风格的带货文案。"),
    ("human", "商品链接或信息如下：\n{product_info}\n\n请严格按照 AffiliatePost 的结构化格式输出。")
])
```

---

## 🕸 终极拼图：加上 LangGraph 和工具 (Tools)

现在，我们要让它变成一个真正的 **Agent 智能体**。

在真实的推客业务中，你肯定不想把精力浪费在低佣金的商品上。所以我们为 Agent 设计这样一个工作流：

1. 用户发来一个 Shopee 商品链接。
2. Agent 自动调用 `check_commission` (查佣金) 工具。
3. 如果佣金率 `< 10%`，Agent 会回复：“这个商品佣金太低，不建议推广”。
4. 如果佣金率 `>= 10%`，Agent 才会进入刚才的文案生成环节。

还记得我们之前讨论过，**工具的描述（Description）对大模型来说极其重要吗？**

假设我们现在要定义这个 `check_commission` 工具，为了确保大模型在看到用户发来商品链接时，**第一反应**是先去调用这个工具查佣金，而不是直接开始瞎编文案，你会怎么写这个工具的 `description` 呢？你可以试着用一两句话描述一下。

支持查询佣金率。适合在有商品链接，文案生成之前调用

太棒了！你的描述非常精准地抓住了大模型工具描述的核心要领：**明确功能 + 划定触发时机**。

对于大模型来说，“在文案生成之前调用”这句话简直就像是一道“圣旨”，能有效地阻止它一上来就凭空捏造文案。

为了让大模型执行得更像一个精密的机器，我们在实际开发中，通常会把你的描述再稍微加上一点“强制性”的语气，并明确输入和输出的格式。比如用 LangChain 的 `@tool` 装饰器，我们会这样写：

Python

代码块

```
from langchain_core.tools import tool

@tooldef check_commission(product_url: str) -> float:"""
    核心工具：查询商品佣金率。
    触发条件：当用户提供商品链接时，【必须】在生成任何推广文案之前调用此工具！
    输入：商品链接 (URL)
    输出：该商品的佣金比例（例如返回 0.12 表示 12%）。
    """# 假设这里调用了你们平台的真实 API
    print(f"正在查询链接的佣金：{product_url}")
    # 模拟数据：假设查询到佣金为 15%return 0.15
```

---

## 🌟 最终大组装：推客联盟智能体架构

现在，我们将你定义的 **数据模型 (`AffiliatePost`)**、**提示词模板**，以及 **工具 (`check_commission`)** 全部放进 LangGraph 的大框架里。

在这个图结构中，核心的路由逻辑（Conditional Edge）就像一个尽职尽责的质检员：

Python

代码块

```
# 这是一个简化的 LangGraph 核心路由逻辑演示def route_based_on_commission(state):"""判断佣金是否达标，决定下一步去哪"""# 假设从状态 (State) 中获取了工具返回的佣金率
    commission_rate = state.get("commission_rate", 0)

    if commission_rate < 0.10:
        # 佣金低于 10%，不值得推，直接走向结束节点return "reject_and_end"else:
```

```
        # 佣金达标，走向文案生成节点，并按照 AffiliatePost 格式输出return "generate_post"# 在 Graph 中绑定这条边
workflow.add_conditional_edges(
    "check_tool_node",      # 当前所在节点：工具查询完毕
    route_based_on_commission, # 路由判断函数
    {
        "reject_and_end": END,                # 结束"generate_post": "copywriter_node"    # 去写文案
    }
)
```

## 🎉 结语

恭喜你！通过这几轮的拆解，你已经完整推演了一个针对东南亚电商联盟的 AI 智能体开发流程。

从最开始大模型的**“孤立无援”，**到用 LangChain 接入**模板与格式（LCEL & BaseModel），**再到挂载**领域知识（RAG），**最后通过 LangGraph 赋予它**查询佣金和自主决策（Agents & Tools）**的能力。你现在已经不仅停留在“怎么用 AI 写段话”的阶段，而是掌握了如何用工程化的思维去“构建一个能处理业务逻辑的 AI 系统”。

接下来，你就可以在你的开发环境里 `pip install langchain langgraph`，把我们讨论的这些片段变成真正的代码跑起来了！